

PROYECTO INTERMODULAR

BASE DE DATOS – PROGRAMACIÓN

Persistencia en un juego de rol por turnos (Witcher RPG)

3^{er} Trimestre

Nombre del alumno/a: Aarón Hernández Moreno y Víctor de Lucas Fernández

Curso académico: 1º DAM G2

Tutor del proyecto: Carlos Galarón

ÍNDICE PAGINADO

1. JUSTIFICACIÓN DEL PROYECTO

2. INTRODUCCIÓN

3. OBJETIVOS

- 3.1. Objetivo general
- 3.2. Objetivos específicos
- 3.3. Objetivos funcionales
- 3.4. Objetivos del proyecto del 3er trimestre

4. DESARROLLO

- 4.1. Fundamentación teórica
- 4.2. Materiales y métodos
- 4.3. Resultados y análisis

5. CASOS DE USO

6. DIAGRAMA

7. CONCLUSIONES

8. LÍNEAS DE INVESTIGACIÓN FUTURAS

9. BIBLIOGRAFÍA



1. JUSTIFICACIÓN DEL PROYECTO

Durante el segundo trimestre desarrollamos en la asignatura de Programación un juego de rol por turnos ambientado en el universo de The Witcher. El juego cuenta con un sistema de combate entre dos equipos de tres personajes cada uno, mecánicas de armas, hechizos y estados alterados, todo gestionado en memoria mediante objetos de Java.

El problema de ese diseño inicial es que toda la información desaparecía al cerrar el programa: cada vez que se ejecutaba el juego, los personajes volvían a su estado inicial y no quedaba ningún registro de las partidas jugadas. Para un juego real esto no es aceptable, ya que el jugador espera poder guardar su progreso y retomarlo más adelante.

Este proyecto intermodular surge precisamente de esa necesidad: dotar al juego de persistencia mediante una base de datos relacional en MySQL. La justificación es doble. Por un lado, académica, ya que integra los contenidos de las asignaturas de Bases de Datos y Programación en un único proyecto coherente. Por otro lado, práctica, porque reproduce un escenario habitual en el desarrollo de software profesional: separar la lógica de negocio de la capa de persistencia de datos.

La elección de MySQL como sistema gestor de base de datos se justifica por ser uno de los más utilizados en el ámbito profesional, por su gratuidad, por su amplia documentación y por su integración sencilla con Java a través del conector JDBC.

2. INTRODUCCIÓN

La presente memoria documenta el diseño e implementación de la base de datos del proyecto Witcher RPG, así como su integración con el código Java del juego. El objetivo es que el juego sea capaz de guardar el estado completo de una partida, cargarlo posteriormente, listar las partidas disponibles y borrar las que ya no se deseen conservar, todo ello mediante operaciones CRUD (Create, Read, Update, Delete).

El juego enfrenta a dos equipos: el bando bueno, formado por Geralt de Rivia, Yennefer de Vengerberg y Ciri; y el bando malo, La Cacería Salvaje, formado por Imlerith, Caranthir y Eredin. Cada personaje pertenece a una de tres clases (Guerrero, Mago o Sacerdote), cada una con mecánicas propias, y dispone de dos armas y uno o varios hechizos.

La base de datos debe reflejar fielmente esta estructura de objetos. Esto plantea un reto interesante de modelado: trasladar una jerarquía de herencia de Java (la clase abstracta Personajes con sus subclases) y unas relaciones de muchos a muchos (un personaje tiene varias armas, varios hechizos y participa en varios combates) a un modelo relacional normalizado.

A lo largo de la memoria se describe la fundamentación teórica del modelo, las herramientas y la metodología empleadas, y los resultados obtenidos, incluyendo el script SQL completo, las operaciones CRUD y la correspondencia entre las clases Java y las tablas de la base de datos.

3. OBJETIVOS

3.1. Objetivo general

Diseñar e implementar una base de datos relacional en MySQL que permita dotar de persistencia al juego Witcher RPG, integrándola con el código Java mediante JDBC para guardar, cargar, listar y borrar partidas.

3.2. Objetivos específicos

- Modelar mediante un diagrama entidad-relación las entidades del juego (personajes, armas, hechizos, estados y combates) y sus relaciones.
- Traducir la jerarquía de herencia de las clases de personajes (Guerrero, Mago, Sacerdote) a tablas relacionales mediante la técnica de tabla por subtipo.
- Resolver las relaciones de muchos a muchos mediante tablas intermedias (personaje-arma, personaje-hechizo, combate-personaje).
- Implementar las operaciones CRUD necesarias para gestionar partidas guardadas.
- Poblar la base de datos con los datos del catálogo del juego (personajes, armas y hechizos con sus estadísticas reales).
- Garantizar la integridad referencial de los datos mediante claves primarias, foráneas y restricciones.
- Documentar la correspondencia entre las clases Java y las tablas de la base de datos.

3.3. Objetivos funcionales

Los siguientes objetivos funcionales, recogidos en el enunciado del proyecto intermodular, se han cumplido totalmente. El juego es capaz de realizar todas las operaciones de persistencia exigidas a través de un menú durante su ejecución:

Objetivo funcional	Estado
Realizar operaciones CRUD a través de un menú durante la ejecución del programa.	✓ Cumplido totalmente
Comenzar una nueva partida: crear un personaje nuevo o retomar uno existente.	✓ Cumplido totalmente
Guardar automáticamente después de cada turno.	✓ Cumplido totalmente
Cargar una partida guardada anteriormente según el turno.	✓ Cumplido totalmente
Borrar partidas que ya no se quieran conservar.	✓ Cumplido totalmente
Listar las partidas disponibles para cargar.	✓ Cumplido totalmente

3.4. Objetivos del proyecto del 3er trimestre

El proyecto parte del código desarrollado en el segundo trimestre y lo amplía con persistencia en base de datos, mejoras en la lógica del juego y nuevos módulos. Las siguientes características, recogidas en el enunciado del proyecto, se han cumplido totalmente:

Característica del enunciado	Estado
Persistencia de partida: guardar y cargar en BD el estado completo de una partida y restaurarlo fielmente.	✓ Cumplido totalmente
Ranking global: clasificación de jugadores en BD ordenada por criterios y actualizada automáticamente.	✓ Cumplido totalmente
Historial de partidas: registro en BD de las acciones relevantes de cada combate, consultable después.	✓ Cumplido totalmente
Logros: sistema de logros persistente con condiciones de desbloqueo y sin duplicidades.	✓ Cumplido totalmente
Guardado automático: almacenamiento del estado en momentos clave (tras cada turno) garantizando la integridad.	✓ Cumplido totalmente
Dificultad configurable: valores almacenados en BD que modifican el comportamiento del juego sin tocar el código.	✗ No realizado
Corrección de lo comentado en clase: refactorizaciones, separación de responsabilidades y buenas prácticas.	✓ Cumplido totalmente
Tests con JUnit (obligatorio): pruebas unitarias de la lógica de negocio	✓ Cumplido

Característica del enunciado	Estado
con casos normales, límite y de error.	totalmente
Visualización de datos con XChart (punto extra): gráficos integrados mediante Maven en el pom.xml.	✓ Cumplido totalmente

4. DESARROLLO

4.1. Fundamentación teórica

El modelo de datos se construyó a partir del análisis del código Java del juego. A continuación se describen las decisiones de diseño más relevantes.

Modelo entidad-relación

Se identificaron cinco entidades principales que se corresponden con las clases del juego: PERSONAJES, ARMAS, HECHIZOS, ESTADOS y COMBATE. Cada una se traduce en una tabla con sus atributos propios.

Tratamiento de la herencia

En el código, la clase abstracta Personajes tiene tres subclases: Guerrero, Mago y Sacerdote, cada una con atributos exclusivos (el Guerrero tiene defensa por adrenalina y probabilidad de bloqueo; el Mago tiene un multiplicador mágico; el Sacerdote tiene un bonus de sanación). Para representar esta herencia en el modelo relacional se utilizó la técnica de tabla por subclase: existe una tabla PERSONAJES con los atributos comunes y tres tablas adicionales (GUERRERO, MAGO, SACERDOTE) cuya clave primaria es a la vez clave foránea hacia PERSONAJES. De este modo, cada personaje tiene una fila en PERSONAJES y otra en la tabla de su clase.

Relaciones de muchos a muchos

El análisis del código reveló tres relaciones de muchos a muchos que no pueden representarse directamente en un modelo relacional y requieren tablas intermedias:

- **PERSONAJE_ARMA:** un personaje dispone de varias armas y cada arma puede pertenecer a varios personajes. El atributo equipada indica cuál arma lleva equipada actualmente (en Java es la diferencia entre la lista de armas disponibles y el arma equipada).
- **PERSONAJE_HECHIZO:** relaciona personajes con hechizos. El atributo cooldownActual guarda los turnos que le quedan a cada hechizo para volver a estar disponible, que en Java se almacenaba en un Map de cooldowns.

- **COMBATE_PERSONAJE:** resuelve la relación entre combates y personajes. Un combate guardado contiene seis personajes y un personaje puede aparecer en varios combates guardados.

Discriminadores en lugar de subtipos para armas y hechizos

Tanto las armas (cuerpo a cuerpo / a distancia) como los hechizos (daño directo, daño en el tiempo, curación directa, curación en el tiempo) tienen subtipos en el código Java. Sin embargo, esos subtipos no añaden atributos nuevos que haya que persistir: solo cambian el comportamiento de un método (polimorfismo). Por ello, en la base de datos no se crearon tablas de subtipo, sino que se utilizó un campo discriminador de tipo ENUM (el campo tipo en ARMAS y tipoHechizo en HECHIZOS) que indica de qué subtipo se trata. Al cargar desde la base de datos, el código usa ese campo para instanciar la subclase correcta.

Datos estáticos frente a datos dinámicos

Una decisión de diseño importante fue distinguir entre los datos que forman parte del catálogo del juego y los que se generan durante la partida. Las tablas PERSONAJES, ARMAS, HECHIZOS y sus relaciones contienen datos estáticos que se insertan al crear la base de datos. En cambio, las tablas COMBATE, COMBATE_PERSONAJE y ESTADOS contienen datos dinámicos que el juego va creando, actualizando y borrando durante su ejecución, por lo que arrancan vacías.

4.2. Materiales y métodos

Para el desarrollo del proyecto se utilizaron las siguientes herramientas y tecnologías:

- **MySQL:** sistema gestor de base de datos relacional donde se crean las tablas y se almacenan los datos.
- **phpMyAdmin:** interfaz web para administrar la base de datos, ejecutar consultas y exportar el script SQL.
- **Java (Eclipse IDE):** lenguaje y entorno donde está programado el juego.
- **Conector JDBC (mysql-connector-j):** librería que permite a Java conectarse a MySQL, gestionada mediante Maven en el archivo pom.xml.
- **Draw.io:** para diseñar el modelo entidad-relación antes de crear las tablas.

Metodología

La metodología seguida fue incremental. En primer lugar se analizó el código Java existente para identificar las entidades y relaciones. A continuación se elaboró el

diagrama entidad-relación, que se fue refinando hasta corregir todas las inconsistencias detectadas (relaciones mal definidas, falta de cardinalidades, subtipos innecesarios). Después se tradujo el diagrama a sentencias CREATE TABLE en SQL y se poblaron las tablas estáticas con los datos del catálogo. Finalmente se definieron las operaciones CRUD que el juego ejecutará para gestionar las partidas.

Estructura de la base de datos

La base de datos consta de las siguientes tablas, agrupadas según su función:

Tabla	Tipo	Función
PERSONAJES	Principal	Datos comunes de los 6 personajes
GUERRERO / MAGO / SACERDOTE	Subtipo (ISA)	Atributos exclusivos de cada clase
ARMAS	Principal	Las 12 armas del juego
HECHIZOS	Principal	Los 10 hechizos del juego
ESTADOS	Principal (dinámica)	Estados activos (Quemadura, Veneno, Renovar)
COMBATE	Principal (dinámica)	Partidas guardadas
PERSONAJE_ARMA	Intermedia	Qué armas tiene cada personaje y cuál equipa
PERSONAJE_HECHIZO	Intermedia	Qué hechizos tiene cada personaje y su cooldown
COMBATE_PERSONAJE	Intermedia (dinámica)	Qué personajes participan en cada combate
JUGADORES	Principal (dinámica)	Datos del jugador: victorias, derrotas y contadores
HISTORIAL	Principal (dinámica)	Acciones registradas de cada combate
LOGROS	Principal (estática)	Catálogo de los 13 logros disponibles
LOGROS_JUGADOR	Intermedia (dinámica)	Qué logros ha desbloqueado cada jugador

4.3. Resultados y análisis

El resultado del proyecto es la base de datos proyecto_rpg, exportada desde phpMyAdmin como un script SQL completo. A continuación se muestran sus elementos

organizados por bloques: la creación de las tablas, la inserción de los datos del catálogo, las claves y el AUTO_INCREMENT, las restricciones de integridad y, por último, las operaciones CRUD que el juego ejecuta desde Java.

Creación de las tablas (CREATE TABLE)

La estructura de la base de datos se define con sentencias CREATE TABLE. La tabla principal PERSONAJES guarda los atributos comunes a todos los personajes, y cada clase (Guerrero, Mago, Sacerdote) tiene su propia tabla de subtipo que comparte la clave con PERSONAJES. Los tipos especiales (cuerpo a cuerpo / a distancia, tipos de hechizo, tipos de estado) se modelan con campos ENUM.

```
CREATE TABLE `personajes` (  
  `ID_personaje` int(11) NOT NULL,  
  `nombre` varchar(100) NOT NULL,  
  `vidaMax` int(11) NOT NULL,  
  `vidaActual` int(11) NOT NULL,  
  `recursoMax` int(11) NOT NULL,  
  `recursoActual` int(11) NOT NULL,  
  `ataqueBase` int(11) NOT NULL,  
  `defensaBase` int(11) NOT NULL,  
  `poderMagico` int(11) NOT NULL DEFAULT 0  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;  
  
CREATE TABLE `guerrero` (  
  `ID_personaje` int(11) NOT NULL,  
  `defensaAdrenalina` int(11) NOT NULL,  
  `probBloqueo` double NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;  
  
CREATE TABLE `armas` (  
  `ID_ARMA` int(11) NOT NULL,  
  `nombre` varchar(100) NOT NULL,  
  `tipo` enum('CUERPO_A_CUERPO','A_DISTANCIA') NOT NULL,  
  `danoBase` int(11) NOT NULL,  
  `modificador` double NOT NULL,  
  `probCritico` double NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;  
  
CREATE TABLE `hechizos` (  
  `ID_HECHIZO` int(11) NOT NULL,  
  `nombre` varchar(100) NOT NULL,  
  `tipoObjetivo` enum('ENEMIGO_UNICO','ALIADO_UNICO') NOT NULL,  
  `tipoHechizo` enum('DANO_DIRECTO','DANO_EN_TIEMPO',  
    'CURACION_DIRECTA','CURACION_EN_TIEMPO') NOT NULL,  
  `potenciaBase` int(11) NOT NULL DEFAULT 0,  
  `costeMana` int(11) NOT NULL,  
  `cooldownMaximo` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Las tablas intermedias resuelven las relaciones de muchos a muchos, y las tablas dinámicas (COMBATE, ESTADOS, HISTORIAL) guardan la información que se genera durante las partidas:

```
CREATE TABLE `personaje_arma` (
  `ID_personaje` int(11) NOT NULL,
  `ID_ARMA` int(11) NOT NULL,
  `equipada` tinyint(1) NOT NULL DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE `personaje_hechizo` (
  `ID_personaje` int(11) NOT NULL,
  `ID_HECHIZO` int(11) NOT NULL,
  `cooldownActual` int(11) NOT NULL DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE `combate` (
  `ID_COMBATE` int(11) NOT NULL,
  `turno` int(11) NOT NULL DEFAULT 1,
  `nRondas` int(11) NOT NULL DEFAULT 0,
  `resumenFinal` varchar(255) DEFAULT NULL,
  `fechaGuardado` datetime NOT NULL DEFAULT current_timestamp(),
  `ID_jugador` int(11) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE `estados` (
  `ID_ESTADO` int(11) NOT NULL,
  `ID_personaje` int(11) NOT NULL,
  `nombre` varchar(50) NOT NULL,
  `turnosRestantes` int(11) NOT NULL,
  `potenciaPorTurno` int(11) NOT NULL,
  `tipoEstado` enum('DOT','HOT') NOT NULL,
  `aplicadoPorSacerdote` tinyint(1) NOT NULL DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Inserción de los datos del catálogo (INSERT)

Una vez creadas las tablas, se rellenan las que contienen datos estáticos del juego con sentencias INSERT. Estos son los seis personajes y las doce armas del catálogo:

```
INSERT INTO `personajes` (`ID_personaje`, `nombre`, `vidaMax`,
`vidaActual`,
`recursoMax`, `recursoActual`, `ataqueBase`, `defensaBase`,
`poderMagico`) VALUES
(1, 'Geralt de Rivia', 800, 0, 100, 10, 85, 30, 0),
(2, 'Yennefer de Vengerberg', 400, 0, 300, 20, 30, 10, 100),
(3, 'Ciri', 600, 0, 200, 30, 70, 20, 50),
(4, 'Imlerith', 1000, 440, 50, 10, 90, 35, 0),
(5, 'Caranthir', 500, 33, 280, 30, 25, 10, 90),
(6, 'Eredin', 750, 40, 200, 30, 60, 25, 60);

INSERT INTO `armas` (`ID_ARMA`, `nombre`, `tipo`, `danoBase`,
`modificador`, `probCritico`) VALUES
(1, 'Aerondight', 'CUERPO_A_CUERPO', 80, 1.2, 0.2),
(2, 'Ballesta Ursina', 'A_DISTANCIA', 50, 1, 0.15),
(5, 'Zireael', 'CUERPO_A_CUERPO', 70, 1.1, 0.3),
(7, 'Maza Gigante', 'CUERPO_A_CUERPO', 100, 1.3, 0.15),
(10, 'Orbe de Escarcha', 'A_DISTANCIA', 65, 1.2, 0.25);
-- (... hasta 12 armas)
```

Las tablas de subtipo guardan los atributos exclusivos de cada clase, y las tablas intermedias asignan a cada personaje sus armas y hechizos (el campo equipada marca con un 1 el arma activa):

```
INSERT INTO `guerrero` (`ID_personaje`, `defensaAdrenalina`,  
`probBloqueo`) VALUES  
(1, 20, 0.2),  
(4, 30, 0.15);  
  
INSERT INTO `mago` (`ID_personaje`, `multiplicadorMagico`) VALUES  
(2, 1.5),  
(5, 1.3);  
  
INSERT INTO `sacerdote` (`ID_personaje`, `bonusSanacion`) VALUES  
(3, 20),  
(6, 15);  
  
INSERT INTO `personaje_arma` (`ID_personaje`, `ID_ARMA`, `equipada`) VALUES  
(1, 1, 0), (1, 2, 1),  
(2, 3, 0), (2, 4, 1),  
(3, 5, 1), (3, 6, 0);  
-- (... resto de personajes)
```

También se carga el catálogo de los trece logros disponibles, que después se vincularán a cada jugador en la tabla LOGROS_JUGADOR:

```
INSERT INTO `logros` (`ID_logro`, `nombre`, `descripcion`) VALUES  
(1, 'PRIMERA_SANGRE', 'Gana tu primer combate.'),  
(2, 'VETERANO', 'Gana 10 combates.'),  
(5, 'GOLPE_CRITICO', 'Asesta un golpe critico con un arma.'),  
(10, 'PIROMANO', 'Aplica Quemadura 20 veces.'),  
(13, 'VENCER_AL_REY', 'Derrota a Eredin.');  
-- (... 13 logros en total)
```

Claves primarias y AUTO_INCREMENT

Tras crear las tablas, el script define las claves primarias con ALTER TABLE. Las tablas principales tienen una clave simple, mientras que las intermedias usan una clave compuesta por las dos claves foráneas, lo que impide que se repita la misma combinación. Además, los identificadores de las tablas principales se configuran como AUTO_INCREMENT para que MySQL los genere automáticamente:

```
ALTER TABLE `personajes`  
  ADD PRIMARY KEY (`ID_personaje`);  
  
ALTER TABLE `personaje_arma`  
  ADD PRIMARY KEY (`ID_personaje`, `ID_ARMA`);  
  
ALTER TABLE `combate_personaje`  
  ADD PRIMARY KEY (`ID_COMBATE`, `ID_personaje`);  
  
ALTER TABLE `armas`  
  MODIFY `ID_ARMA` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=13;  
  
ALTER TABLE `combate`  
  MODIFY `ID_COMBATE` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=9;
```

Restricciones de integridad (FOREIGN KEY)

Por último se añaden las claves foráneas, que garantizan la integridad referencial: una fila de una tabla de subtipo o intermedia no puede existir sin su personaje, arma, hechizo o combate correspondiente. Estas restricciones son también las que obligan a respetar un orden concreto al borrar datos.

```
ALTER TABLE `guerrero`  
  ADD CONSTRAINT `guerrero_ibfk_1`  
  FOREIGN KEY (`ID_personaje`) REFERENCES `personajes` (`ID_personaje`);  
  
ALTER TABLE `personaje_arma`  
  ADD CONSTRAINT `personaje_arma_ibfk_1`  
  FOREIGN KEY (`ID_personaje`) REFERENCES `personajes` (`ID_personaje`),  
  ADD CONSTRAINT `personaje_arma_ibfk_2`  
  FOREIGN KEY (`ID_ARMA`) REFERENCES `armas` (`ID_ARMA`);  
  
ALTER TABLE `combate_personaje`  
  ADD CONSTRAINT `combate_personaje_ibfk_1`  
  FOREIGN KEY (`ID_COMBATE`) REFERENCES `combate` (`ID_COMBATE`),  
  ADD CONSTRAINT `combate_personaje_ibfk_2`  
  FOREIGN KEY (`ID_personaje`) REFERENCES `personajes` (`ID_personaje`);
```

Operaciones CRUD desde el juego

El script anterior crea y puebla la base de datos. Las operaciones CRUD que se ejecutan durante la partida no forman parte del volcado SQL, sino que las lanza el código Java mediante JDBC (clase ConexionBD con PreparedStatement) en los momentos correspondientes del juego. Todas las consultas que siguen están tomadas directamente del código fuente, principalmente de la clase PersistenciaPartida. La siguiente tabla resume cuándo ocurre cada operación:

Operación	Cuándo ocurre en el juego	Tablas afectadas
CREATE	Al registrar un jugador, crear una partida, aplicar un estado o registrar el	JUGADORES, COMBATE, COMBATE_PERSONAJE, ESTADOS,

Operación	Cuándo ocurre en el juego	Tablas afectadas
	historial	HISTORIAL
READ	Al listar partidas, cargar una guardada o mostrar el ranking	COMBATE, PERSONAJES, PERSONAJE_ARMA, PERSONAJE_HECHIZO, ESTADOS, JUGADORES
UPDATE	Tras cada turno y al finalizar la partida (vida, recurso, cooldowns, arma, resultado)	PERSONAJES, PERSONAJE_ARMA, PERSONAJE_HECHIZO, COMBATE, JUGADORES
DELETE	Al borrar una partida o al reescribir los estados de un personaje	ESTADOS, COMBATE_PERSONAJE, HISTORIAL, COMBATE

CREATE. Al empezar, se registra el jugador (si no existía) y se crea el combate con sus seis personajes. También se insertan los estados aplicados y las acciones del historial:

```
-- Registrar un jugador nuevo (PersistenciaPartida.obtenerOCrearJugador)
INSERT INTO jugadores (nombre) VALUES (?);

-- Crear una partida nueva (PersistenciaPartida.nuevaPartida)
INSERT INTO COMBATE (turno, nRondas, ID_jugador) VALUES (1, 0, ?);
INSERT INTO COMBATE_PERSONAJE (ID_COMBATE, ID_personaje) VALUES (?, ?);

-- Guardar un estado aplicado (PersistenciaPartida.guardarEstados)
INSERT INTO ESTADOS (ID_personaje, nombre, turnosRestantes,
                    potenciaPorTurno, tipoEstado, aplicadoPorSacerdote)
VALUES (?, ?, ?, ?, ?, ?);

-- Registrar una accion en el historial (GestorHistorial.registrar)
INSERT INTO HISTORIAL (ID_COMBATE, ronda, accion) VALUES (?, ?, ?);
```

READ. Al cargar una partida se leen la ronda y el jugador del combate y, para cada personaje, su vida, recurso, arma equipada, cooldowns y estados. El ranking usa un JOIN entre combate y jugadores:

```
-- Datos del combate guardado (PersistenciaPartida.cargarEstado)
SELECT nRondas, ID_jugador FROM COMBATE WHERE ID_COMBATE=?;

-- Estado de cada personaje
SELECT vidaActual, recursoActual FROM PERSONAJES WHERE ID_personaje=?;
SELECT equipada FROM PERSONAJE_ARMA WHERE ID_personaje=? ORDER BY
ID_ARMA;
SELECT cooldownActual FROM PERSONAJE_HECHIZO
WHERE ID_personaje=? ORDER BY ID_HECHIZO;
SELECT nombre, turnosRestantes, potenciaPorTurno, aplicadoPorSacerdote
FROM ESTADOS WHERE ID_personaje=?;

-- Listar las partidas guardadas (PersistenciaPartida.listarPartidas)
SELECT ID_COMBATE, nRondas, resumenFinal, fechaGuardado
FROM COMBATE ORDER BY fechaGuardado DESC;

-- Ranking de jugadores (Ranking.mostrarRanking)
SELECT j.nombre, c.nRondas, c.resumenFinal, c.fechaGuardado
FROM combate c
LEFT JOIN jugadores j ON j.ID_jugador = c.ID_jugador
WHERE c.resumenFinal IS NOT NULL
ORDER BY c.fechaGuardado DESC;
```

UPDATE. Después de cada turno se guardan los valores que han cambiado (vida, recurso, arma equipada, cooldowns y número de ronda), y al terminar se marca el resultado de la partida y se actualizan las estadísticas del jugador:

```
-- Guardar el estado tras un turno (PersistenciaPartida.guardarEstado)
UPDATE PERSONAJES SET vidaActual=?, recursoActual=? WHERE
ID_personaje=?;
UPDATE PERSONAJE_ARMA SET equipada=? WHERE ID_personaje=? AND ID_ARMA=?;
UPDATE PERSONAJE_HECHIZO SET cooldownActual=?
WHERE ID_personaje=? AND ID_HECHIZO=?;
UPDATE COMBATE SET nRondas=? WHERE ID_COMBATE=?;

-- Finalizar la partida (PersistenciaPartida.finalizarPartida)
UPDATE COMBATE SET nRondas=?, resumenFinal=? WHERE ID_COMBATE=?;

-- Estadísticas del jugador (PersistenciaPartida / GestorLogros)
UPDATE jugadores SET victorias = victorias + 1 WHERE ID_jugador = ?;
UPDATE jugadores SET derrotas = derrotas + 1 WHERE ID_jugador = ?;
```

DELETE. Antes de volver a guardar los estados de un personaje se borran los anteriores. Al eliminar una partida hay que respetar el orden que imponen las claves foráneas: primero las tablas dependientes (COMBATE_PERSONAJE e HISTORIAL) y, por último, la fila principal de COMBATE:

```
-- Reescribir los estados de un personaje  
(PersistenciaPartida.guardarEstados)  
DELETE FROM ESTADOS WHERE ID_personaje=?;  
  
-- Borrar una partida en el orden correcto  
(PersistenciaPartida.borrarPartida)  
DELETE FROM COMBATE_PERSONAJE WHERE ID_COMBATE=?;  
DELETE FROM HISTORIAL WHERE ID_COMBATE=?;  
DELETE FROM COMBATE WHERE ID_COMBATE=?;
```

Correspondencia entre clases Java y tablas

La siguiente tabla muestra cómo se relaciona cada clase del juego con su tabla correspondiente en la base de datos, que es uno de los entregables exigidos:

Clase Java	Tabla BD	Observaciones
Personajes (abstracta)	PERSONAJES	Atributos comunes a todos
Guerrero	GUERRERO	PK = FK a PERSONAJES
Mago	MAGO	PK = FK a PERSONAJES
Sacerdote	SACERDOTE	PK = FK a PERSONAJES
Armas (+ subtipos)	ARMAS	Subtipo en campo ENUM tipo
Hechizos (+ subtipos)	HECHIZOS	Subtipo en campo ENUM tipoHechizo
Estados (Quemadura, Veneno, Renovar)	ESTADOS	Subtipo en campo ENUM tipoEstado
Combate	COMBATE	Una fila por partida guardada
Map de cooldowns	PERSONAJE_HECHIZO	El Map se traduce en filas
Lista de armas	PERSONAJE_ARMA	equipada distingue la equipada

Integración con Java mediante JDBC

La conexión con la base de datos se centraliza en la clase ConexionBD, que abre la conexión a MySQL (jdbc:mysql://localhost:3306/proyecto_rpg) mediante el conector JDBC declarado en el pom.xml de Maven. Esta clase ofrece tres métodos reutilizables: consultar() para los SELECT (devuelve cada fila como un Object[]), ejecutar() para INSERT, UPDATE y DELETE (devuelve las filas afectadas) y ejecutarYObtenerId() para los INSERT que necesitan recuperar la clave generada con AUTO_INCREMENT. Todas usan sentencias preparadas (PreparedStatement) con el carácter ? como marcador de posición, lo que evita errores de concatenación y protege frente a inyección SQL. Las clases PersistenciaPartida, GestorHistorial, GestorLogros y Ranking se apoyan en

ConexionBD para guardar la partida tras cada turno, cargarla, listarla, borrarla y mantener el historial, el ranking y los logros.

5. CASOS DE USO

A continuación se describen los principales casos de uso que conectan el juego con la base de datos. Cada uno detalla la fuente de datos, el actor, una descripción, el flujo básico y los flujos alternativos que pueden producirse.

5.1. Ejecutar turno manual

Caso de uso	Ejecutar turno manual
Fuentes	Base de datos (PERSONAJES, PERSONAJE_HECHIZO, PERSONAJE_ARMA, ARMAS, HECHIZOS)
Actor	Jugador
Descripción	El jugador controla a un personaje de su equipo durante su turno, eligiendo qué acción realizar (usar hechizo o atacar con arma).
Flujo básico	1. El sistema muestra los hechizos del personaje con su estado (disponible, en cooldown, sin mana). 2. El sistema muestra la opción de atacar con el arma equipada. 3. El jugador elige una opción introduciendo un número. 4. Si elige un hechizo: se lanza sobre un enemigo vivo al azar (si es ofensivo) o sobre el aliado más débil (si es curativo). 5. Si elige atacar con arma: se calcula el daño contra un enemigo vivo al azar. 6. El sistema actualiza la vida del objetivo y el recurso/cooldown del atacante.
Flujo alternativo	3a. El jugador introduce un número fuera de rango → el sistema pide que elija de nuevo. 3b. El jugador introduce texto en vez de un número → el sistema pide que elija de nuevo. 4a. Todos los hechizos están en cooldown o sin mana → el jugador solo puede atacar con arma.

5.2. Atacar con un arma

Caso de uso	Atacar con un arma
Fuentes	Base de datos (PERSONAJES, PERSONAJE_ARMA, ARMAS)
Actor	Jugador (modo manual) / Sistema (modo automático)
Descripción	Un personaje ataca a un enemigo vivo con su arma equipada, calculando el daño según las estadísticas del arma y del personaje.

Flujo básico	1. El sistema selecciona un enemigo vivo al azar. 2. El sistema calcula el daño usando danoBase, modificador y probCritico del arma, y ataqueBase del personaje. 3. El enemigo recibe el daño (reducido por su defensaBase). 4. El sistema muestra el resultado del ataque. 5. Se actualiza la vida del enemigo en la BD.
Flujo alternativo	2a. El ataque es crítico (según probCritico) → el daño se multiplica. 3a. La defensa del enemigo supera el daño → el daño final es 0. 5a. La vida del enemigo llega a 0 → el sistema muestra que ha muerto.

5.3. Lanzar hechizo

Caso de uso	Lanzar hechizo
Fuentes	Base de datos (PERSONAJES, PERSONAJE_HECHIZO, HECHIZOS, ESTADOS)
Actor	Jugador (modo manual) / Sistema (modo automático)
Descripción	Un personaje lanza un hechizo sobre un objetivo, consumiendo mana y activando el cooldown del hechizo.
Flujo básico	1. El sistema verifica que el hechizo está disponible (cooldown = 0 y mana suficiente). 2. Se descuenta el costeMana del recursoActual del personaje. 3. Se activa el cooldown del hechizo (cooldownActual = cooldownMaximo). 4. Si es DAÑO_DIRECTO: se calcula el daño con potenciaBase y poderMagico, y se reduce la vida del enemigo. 5. Si es CURACION_DIRECTA: se cura al aliado más débil con potenciaBase + bonusSanacion (si es sacerdote). 6. Si es DAÑO_EN_TIEMPO: se crea un estado Quemadura sobre el enemigo. 7. Si es CURACION_EN_TIEMPO: se crea un estado Renovar sobre el aliado más débil. 8. El sistema muestra el resultado.
Flujo alternativo	1a. El hechizo está en cooldown → no se puede lanzar, el sistema informa al jugador. 1b. El personaje no tiene mana suficiente → no se puede lanzar. 4a. El daño mágico se amplifica por multiplicadorMagico si el personaje es Mago.

6. DIAGRAMA

El modelo entidad-relación extendido (EER) del proyecto se elaboró en draw.io y pasó por dos versiones. En las páginas siguientes se muestran ambos diseños a tamaño completo para facilitar su lectura.

El primer diseño recoge el núcleo del juego: la entidad PERSONAJES con sus subtipos (Mago, Guerrero y Sacerdote), su relación con ARMAS (EQUIPAR) y HECHIZOS (USAR), los ESTADOS aplicables (Quemadura, Veneno y Renovar) y la entidad COMBATE.

El segundo diseño es la versión más completa y todavía en proceso de finalización. Amplía el modelo anterior incorporando la gestión del jugador y el progreso: las entidades JUGADOR, LOGROS (DESBLOQUEA) y ESTADISTICAS (GUARDA), junto con la tabla intermedia COMBATE_PERSONAJE que resuelve la participación de los personajes en cada combate.

6.1. Primer diseño del modelo EER

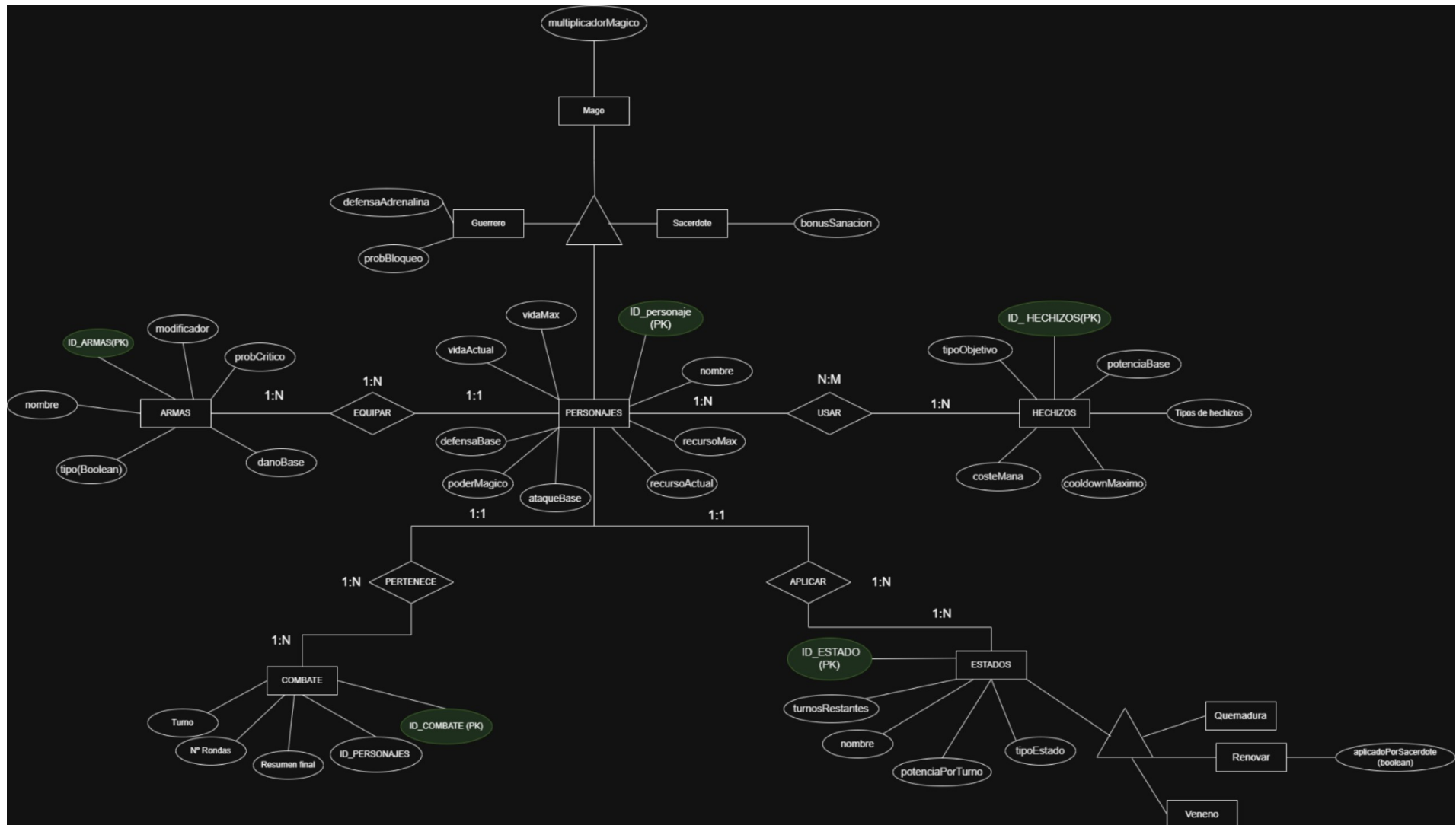


Figura 1. Primer diseño del diagrama entidad-relación (EER) del juego Witcher RPG.

6.2. Segundo diseño del modelo EER (en proceso)

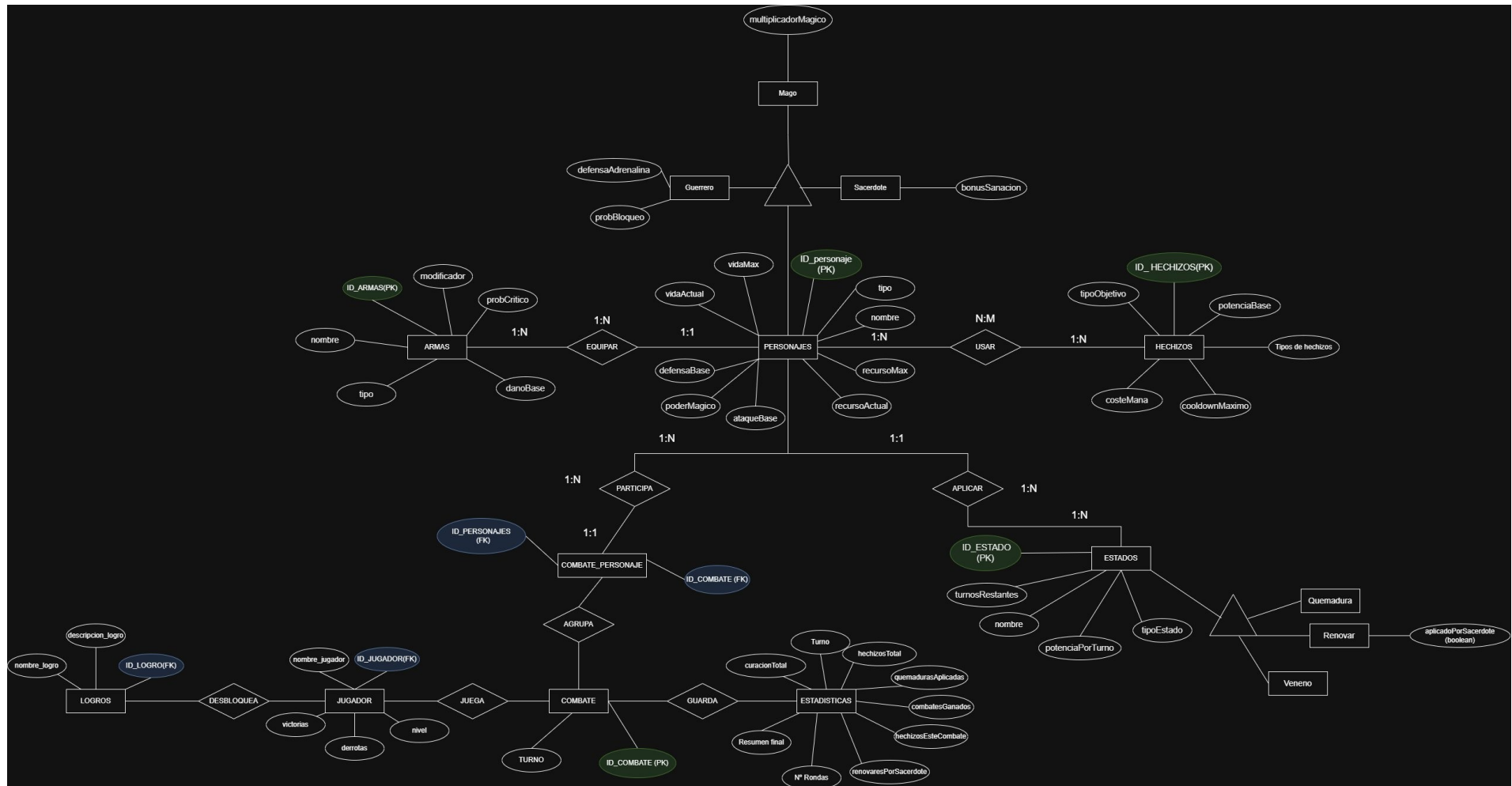


Figura 2. Segundo diseño del diagrama EER, versión ampliada con jugador, logros y estadísticas (pendiente de finalizar).

7. CONCLUSIONES

El proyecto ha cumplido el objetivo de dotar de persistencia al juego mediante una base de datos relacional en MySQL. Se ha conseguido un modelo normalizado que refleja fielmente la estructura de objetos del juego, resolviendo correctamente los retos de modelado que planteaba el código original.

Entre los aprendizajes más importantes destaca la traducción de conceptos de la programación orientada a objetos al modelo relacional: la herencia se resolvió con tablas de subtipo, las colecciones (listas y mapas) se convirtieron en tablas intermedias, y el polimorfismo de los subtipos que no añadían datos se sustituyó por campos discriminadores de tipo ENUM.

También se ha comprendido la importancia de la integridad referencial y del orden de las operaciones, especialmente en los borrados, donde respetar las dependencias entre tablas es imprescindible para no provocar errores.

Por último, la distinción entre datos estáticos (catálogo) y datos dinámicos (partidas) ha resultado clave para diseñar una base de datos coherente, en la que las tablas de partidas arrancan vacías y se van llenando a medida que el jugador juega.

8. LÍNEAS DE INVESTIGACIÓN FUTURAS

A partir de la base actual, el proyecto puede ampliarse en varias direcciones:

- Guardado automático tras cada turno, de modo que el progreso nunca se pierda ante un cierre inesperado del juego.
- Ranking global de jugadores ordenado por victorias o nivel, apoyado en la tabla JUGADORES ya creada.
- Historial detallado de partidas que registre todas las acciones relevantes de cada combate.
- Sistema de dificultad configurable almacenado en base de datos, que modifique la vida o el daño de los enemigos sin tocar el código.
- Persistencia completa del sistema de logros, vinculando los logros desbloqueados a cada jugador en la tabla LOGROS_JUGADOR.

9. BIBLIOGRAFÍA

Documentación oficial de MySQL. <https://dev.mysql.com/doc/>

Documentación oficial de phpMyAdmin. <https://www.phpmyadmin.net/docs/>

Documentación del conector MySQL para Java (JDBC).
<https://dev.mysql.com/doc/connector-j/en/>

Apuntes y materiales de las asignaturas de Bases de Datos y Programación, 1º DAM.

Documento de requisitos del proyecto intermodular proporcionado por el profesorado.